

Sahi Dawa AI: Final Project & QA Summary

1. Project status

Sahi Dawa AI is integrated, tested, and deployed live.

Live application

[Sahi Dawa AI Production App](#)

API documentation

[Swagger API Docs](#)

2. What was originally in the project

Backend

The backend was already substantially complete:

- FastAPI
- Pandas-based medicine catalogue
- Real **131-record medicine catalogue**
- SQLite patient history
- Medicine matching
- Ambiguity detection
- Alternative medicines
- Price comparison
- Pattern detection
- Groq explanation layer
- REST API endpoints
- Pytest test suite

Frontend problem

Originally, there wasn't a proper production frontend.

There was:

- A Gradio application

- A static HTML design/mockup

The static mockup contained its own mock catalogue and **didn't communicate with the real API**, so it couldn't be used as the actual production frontend.

3. Integration work completed

We converted the project into a real integrated application.

Real frontend

Built:

frontend-web/

using:

- Plain HTML
- CSS
- JavaScript
- No frontend framework
- No build process
- No unnecessary dependencies

The frontend is a **thin API client**.

It doesn't calculate or invent medicine information.

Instead:

User



Frontend



FastAPI



131-record verified catalogue



Matching / alternatives / pricing



History / pattern detection



Groq explanation



JSON response



Frontend display

4. Frontend screens implemented

The frontend follows the intended four-stage experience:

01 Prescription

User enters:

- Patient ID
- Diagnosis
- Medicine
- Dosage

02 Results

Displays:

- FOUND
- AMBIGUOUS
- NOT_FOUND

03 Explanation

Displays the backend-generated Groq explanation.

The frontend **doesn't rewrite or generate this explanation**.

04 Summary

Displays patient history and detected discussion flags.

5. Backend fixes

Two important production issues were identified and fixed.

Issue 1 Groq failure

Originally:

Missing GROQ_API_KEY

↓

ExplanationService crash

↓

500 error

This violated the PRD requirement for graceful degradation.

Now:

Groq unavailable



explanation = null



core prescription response still works

So the AI explanation is an enhancement rather than a dependency for the core medicine lookup.

Issue 2 CORS

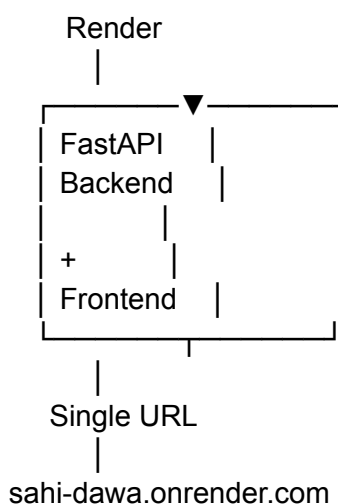
CORS wasn't configured originally.

It was added so the API could safely support a separately hosted frontend.

However, because we ultimately used a **one-service architecture**, the production frontend is served by FastAPI itself, which simplifies deployment considerably.

6. Deployment architecture

Instead of deploying two separate applications, we used:



This gives us:

- One deployment

- One URL
- No frontend/backend domain coordination
- No separate frontend build
- No separate frontend hosting
- Simpler maintenance

7. Deployment files added

The project was prepared with:

- `Dockerfile`
- `render.yaml`
- `.env.example`
- Updated `README.md`

The deployment uses Render's free web-service option.

Important Render limitation

The free service can sleep after inactivity, so the first request after sleeping can take roughly **30–50 seconds**.

That's expected behavior of the free hosting tier, not an application failure.

8. Production API testing

We first tested the backend locally against the actual:

131-record catalogue

The full pytest suite passed:

53/53 tests passing 

Then we tested the deployed production API.

9. Live API test results

TC-01 Health

GET /health

Result:

200

```
{  
  "status": "ok"  
}
```

 PASS

TC-02 Empty submission

Invalid/empty prescription submission.

Expected validation behavior.

 PASS

TC-03 FOUND

Input:

Medicine: Azitron

Dosage: 250mg

Result:

FOUND

Returned:

MED011

Azitron

Azithromycin

250mg

Tablet

6's

PKR 324.33

 PASS

10. Groq explanation

We tested the actual deployed Groq integration.

The live response contained a real:

"explanation": "Our verified catalogue shows..."

The explanation included:

- Medicine information
- Price
- Unit price
- Alternative pricing
- General medical context
- Healthcare-professional disclaimer

Most importantly:

This was generated through the production Groq configuration, not a mocked response.

 PASS

11. AMBIGUOUS test

Input:

Medicine: Azeloc

Dosage: empty

Result:

AMBIGUOUS

Candidates returned:

MED009 — Tablet — 1 x 6's

MED010 — Capsule — 6's

MED012 — Tablet — 1 x 10's

The system correctly **doesn't arbitrarily select one**.

It asks the user to specify/select the exact medicine.

✓ PASS

12. Ambiguous candidate resolution

We selected:

MED010

Result:

Azeloc 250mg Capsule

The frontend correctly performed:

GET /medicine/MED010

GET /alternatives/MED010

and displayed:

- Medicine information
- Price
- Alternatives
- Unit price
- Catalogue status

✓ PASS

13. Nuberol-P ambiguity

Test:

Nuberol-P
120mg/5ml

The API correctly returned:

AMBIGUOUS

with multiple pack volumes:

60ml

90ml
100ml
120ml
450ml

✓ PASS

14. NOT_FOUND

Test:

Panadol

Result:

NOT_FOUND

Backend returned the correct message:

This medicine is not currently available in our verified catalogue.

No fake medicine record or fake price was displayed.

✓ PASS

15. Wrong strength

Test:

Azitron
500mg

while catalogue has:

Azitron
250mg

Result:

NOT_FOUND

The system correctly did **not** substitute the 250mg product.

 PASS

16. Medicine endpoint

Test:

GET /medicine/MED011

Result:

200

Correct Azitron record returned.

 PASS

17. Invalid medicine endpoint

Test:

GET /medicine/MED999

Result:

404

Correct error behavior.

 PASS

18. Alternatives endpoint

Test:

GET /alternatives/MED011

Result:

200

Three verified alternatives returned.

 PASS

19. Invalid alternatives

Test:

GET /alternatives/MED999

Result:

404

 PASS

20. New patient history

Tested an unseen patient.

Result:

200

with empty history rather than 404.

Conceptually:

```
{
  "patient_id": "MANUAL-NEW-001",
  "encounters": [],
  "pattern_flags": []
}
```

 PASS

21. Repeated antibiotic detection

This was one of the most important tests.

Patient:

MANUAL-ANTIBIOTIC-001

Three antibiotic encounters were recorded.

After the third:

REPEATED_ANTIBIOTIC

was correctly triggered.

The UI displayed the discussion warning and explicitly stated that it is:

a discussion flag, not a diagnosis.

 PASS

22. Patient summary

The summary correctly showed:

patient_id

MANUAL-ANTIBIOTIC-001

total_encounters

3

first_encounter_date

2026-09-13

last_encounter_date

2026-09-13

It also showed:

- Throat infection
- Azitron
- Tablet
- 250mg
- Antibiotic classification

- Three recorded encounters
- Repeated-antibiotic flag

The medical disclaimer was also displayed.

✓ PASS

23. Frontend/API integrity

We specifically verified that the application isn't simply displaying the original static mockup.

The application is using the real backend responses.

For example, the production UI displayed the actual:

MED011
Azitron
Azithromycin
PKR 324.33

along with live alternatives and live history.

✓ PASS

24. Overall test results

Automated

53/53 pytest tests passing ✓

Manual functional tests

16/16 completed tests passed ✓

Production API

All major tested endpoints behaved correctly:

/health	✓
/prescription	✓
/medicine/{id}	✓
/alternatives/{id}	✓

/history/{patient} 
/summary/{patient} 

Including expected:

200
404
FOUND
AMBIGUOUS
NOT_FOUND
REPEATED_ANTIBIOTIC

25. Current production verdict

READY FOR DEMO / MVP USE

Component	Status
Backend	 Ready
Database/history	 Working
131-record catalogue	 Working
API endpoints	 Tested
Medicine matching	 Working
Ambiguity detection	 Working
Alternatives	 Working
Price comparison	 Working
Pattern detection	 Working
Groq AI explanation	 Working
Frontend	 Live
Frontend/API integration	 Working
Docker deployment	 Working
Render deployment	 Live
Swagger	 Live

Automated tests  53/53

Manual functional tests  16/16

26. Live URLs

Application

<https://sahi-dawa.onrender.com>

API documentation

[Swagger / OpenAPI Docs](#)

27. What remains

There is **no major blocker** remaining.

The sensible next phase is:

1. Clean up the stale **message** display.
2. Improve any final UI polish.
3. Add persistent production database if this moves beyond demo/MVP usage.
4. Consider paid/non-sleeping hosting when real users arrive.
5. Add authentication/authorization if patient data will be used seriously.
6. Add monitoring/logging.
7. Keep the Groq API key only as a server-side Render secret.

Bottom line

Sahi Dawa AI went from a backend + static frontend mockup to a real integrated application, was tested against the actual 131-record catalogue, passed the 53-test automated suite, passed the manual functional test sequence, successfully generated a real Groq explanation in production, and is now deployed live on Render.